# Mid-Atlantic Regression

## Table of contents

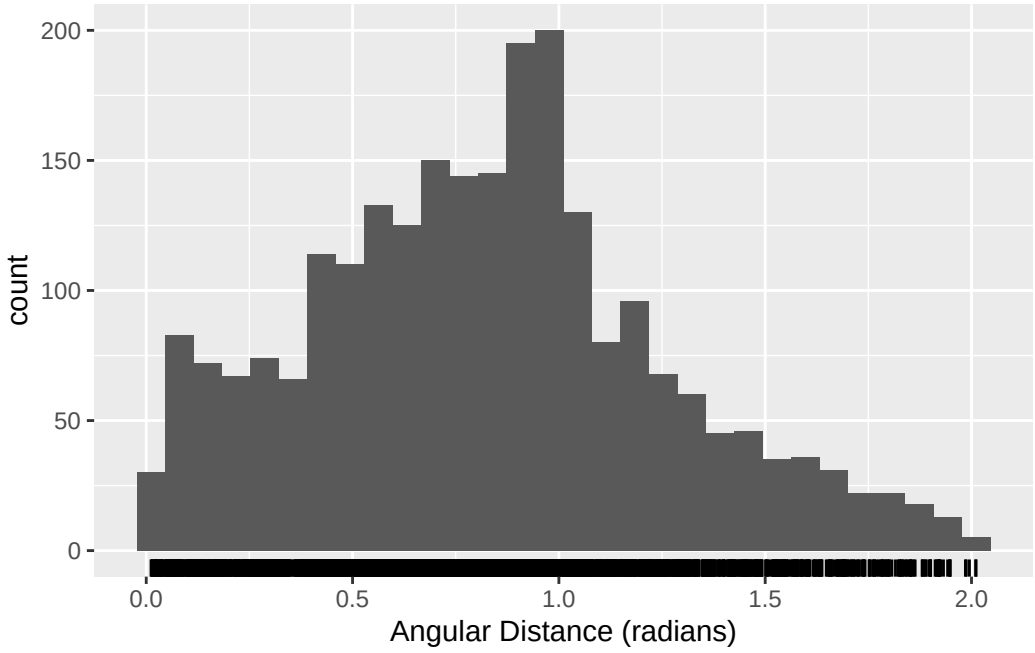
---

**Code version: 0.0.20**

# 1 Data

# 2 Angular Distance Between Observations

```
Y <- fulldf %>%
  select(starts_with("Y")) %>%
  as.matrix()
ang_dist <- acos(pmin(Y %*% t(Y), 1)) # pmin clamps dot products to <=1, preventing NaN from
tibble::enframe(ang_dist[lower.tri(ang_dist)], value = "ang_dist") %>%
  ggplot() +
  geom_histogram(aes(x = ang_dist), bins = 30) +
  geom_rug(aes(x = ang_dist)) +
  scale_x_continuous(name = "Angular Distance (radians)")
```



## 3 Euc Covariates

```
xe <- fulldf %>%
  select(westedge)
xestd <- xe %>% scale() %>% as_tibble()
```

## 4 Our SvMF Model

## 5 Proposition 4 Submodels

The four submodels discussed in the second paragraph of Section 5.1: the constrained links of Proposition 4(i) and 4(ii), each with and without the Euclidean covariate. Proposition 4(ii) fixes $B_s = I_{p-1}$, so the spherical part of the link is a plain rotation $\mu(x_s) = \widetilde{R}_0 x_s$. Proposition 4(i) relaxes that to an isotropic scale $B_s = \beta_s I_{p-1}$ with $0 < \beta_s \leq 1$, so $\beta_s = 1$ recovers Proposition 4(ii).

Proposition 4(i) is stated in the manuscript for $q_e = 0$. The `+ Euclidean` variants below keep the restriction on the spherical component and add the same linear Euclidean term the `LinEuc` model uses. Unlike `mod` above, these are given the *unstandardised* covariate: the Proposition 4 fitters centre and scale it internally and store the transformation on the fitted link.

```
bind_rows(lapply(prop4mods, function(m) {
  tibble(k = m$k, DoF = m$DoF, logLik = m$lLik, AIC = m$AIC)
}), .id = "Model")
```

```
# A tibble: 4 x 5
  Model                                        k   DoF logLik   AIC
  <chr>                                    <dbl> <dbl>  <dbl> <dbl>
1 Proposition 4(i): spherical only          66.3    11  103.  -184.
2 Proposition 4(i): spherical + all Euclidean 488.    15  235.  -440.
3 Proposition 4(ii): spherical only          62.2     8   98.8 -182.
4 Proposition 4(ii): spherical + all Euclidean 476.    14  233.  -439.
```

The isotropic spherical scale estimated by the Proposition 4(i) models. $\beta_s = 1$ corresponds to Proposition 4(ii), and $\phi = (1 - \beta_s)/(1 + \beta_s)$.

```
bind_rows(lapply(prop4mods[1:2], function(m) {
  tibble(beta_s = m$mean$beta_s, phi = m$mean$phi,
         psi_norm = sqrt(sum(m$mean$psi^2)))
}), .id = "Model")
```

```
# A tibble: 2 x 4
  Model                                       beta_s    phi psi_norm
  <chr>                                        <dbl>  <dbl>    <dbl>
1 Proposition 4(i): spherical only             0.872 0.0683   0.0683
2 Proposition 4(i): spherical + all Euclidean  0.929 0.0369   0.0369
```

The estimated scales $a$ of the SvMF error distribution.

```
bind_rows(lapply(prop4mods, function(m) {
  as_tibble_row(setNames(as.list(m$a), paste0("a", seq_along(m$a))))
}), .id = "Model")
```

```
# A tibble: 4 x 4
  Model                                          a1    a2    a3
  <chr>                                       <dbl> <dbl> <dbl>
1 Proposition 4(i): spherical only                1  3.21 0.312
2 Proposition 4(i): spherical + all Euclidean     1  1.96 0.511
3 Proposition 4(ii): spherical only               1  3.17 0.316
4 Proposition 4(ii): spherical + all Euclidean    1  1.95 0.513
```

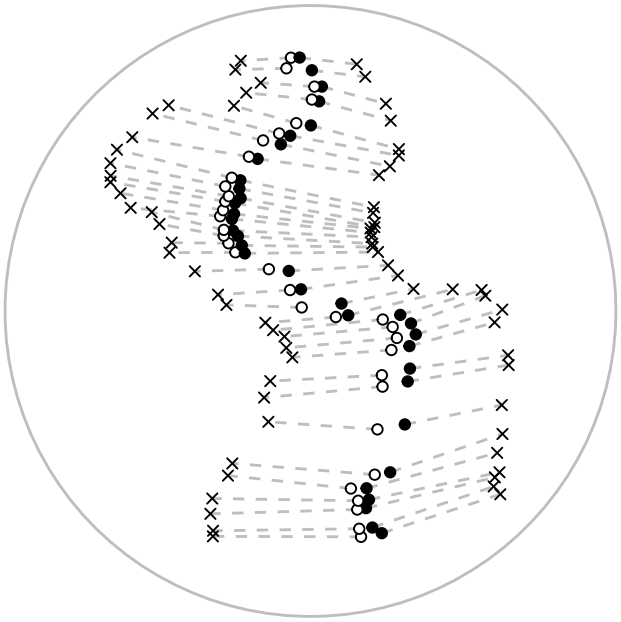
## 6 Pretty Data Plot

```
ymean <- colMeans(fulldf %>% select(starts_with("Y")))
ymean <- ymean/sqrt(sum(ymean^2))
#project to equator
ymeanproj <- c(ymean[1:2], 0)
ymeanproj/vnorm(ymeanproj)
```

```
        Y1         Y2
 0.8791608 -0.4765251  0.0000000
```

```
target <- c(1,0,0)
prettyrot <- parallel_transport_mat(ymeanproj, target)
prettyY <- (fulldf %>%
  select(Y1, Y2, Y3) %>%
  as.matrix()) %*% t(prettyrot)
colnames(prettyY) <- paste0("Y", 1:3)
prettyX <- (fulldf %>%
```

```
  select(X1, X2, X3) %>%
  as.matrix()) %*% t(prettyrot)
colnames(prettyX) <- paste0("X", 1:3)
prettyXmean <- colMeans(prettyX)
prettyXmean <- prettyXmean/vnorm(prettyXmean)
# denomloc <- drop(prettyrot %*% as_mobius_link_cann(mod$mean)$Qs[,1])
plotdata <- bind_cols(prettyY, prettyX, westedge = fulldf$westedge) %>%
  as_tibble() %>%
  ggplot() +
  geom_segment(aes(x=X2, y=X3, xend=Y2, yend=Y3), lty = "dashed", col = "grey") +
  geom_point(aes(x=Y2, y=Y3), shape = 4) +
  geom_point(aes(x=X2, y=X3, fill = westedge), show.legend = FALSE, shape = 21) +
  geom_path(data = circle_df, aes(x = x, y = y), inherit.aes = FALSE, color = "grey") +
  scale_fill_manual(values = c("black", "white")) +
  theme_void() +
  coord_fixed()
  # mark mean of spherical covariate (obtained earlier)
  # annotate("point", x = prettyXmean[2], y = prettyXmean[3], shape = 24, size = 2)
  # annotate("point", x = denomloc[2], y = denomloc[3], shape = 3, size = 4, col = "blue")
plotdata
```



## 7 Predictions

The below 'ours' model is using LinEuc's link with an extra covariate for an intercept and G01 free. The results here are from a local optimisation using gradient from default starting values. I've checked that global search for the best vMF regression does not find anything better.

```
cann <- as_mobius_link_cann(mod$mean)
cann
```

```
$P
         [,1]       [,2]        [,3]
Y1  0.4913098  0.5626852 -0.66483080
Y2 -0.4803628 -0.4616721 -0.74572813
Y3 -0.7265440  0.6857436  0.04346904

$Bs
          [,1]      [,2]
[1,] 0.9029649 0.0000000
[2,] 0.0000000 0.7702704

$Qs
         [,1]       [,2]       [,3]
X1  0.2657933  0.5868954 -0.7647926
X2 -0.5034120 -0.5920730 -0.6293059
X3 -0.8221498  0.5522710  0.1380812

$Be
           [,1]      [,2]
[1,] 0.07230452 0.0000000
[2,] 0.00000000 0.2525752

$Qe
          [,1]       [,2]      [,3]
dummyzero    1  0.0000000 0.0000000
westedge     0  0.1254316 0.9921023
ones         0 -0.9921023 0.1254316

$ce
[1] 1

attr(,"class")
[1] "mobius_link_cann" "list"
```

Lets try to interpret the fitted link.

```
cann$Bs %*% t(cann$Qs[,-1])
```

```
             X1         X2        X3
[1,]  0.5299460 -0.5346212 0.4986814
[2,] -0.5890971 -0.4847357 0.1063598
```

```
cann$Be %*% t(cann$Qe[,-1])
```

```
     dummyzero    westedge        ones
[1,]         0 0.009069271 -0.07173348
[2,]         0 0.250580391  0.03168090
```

The first direction away from $B_{01}$ (first row in above) is roughly equally influenced by X1, X2 and X3 (all are about 0.5) with west edge having very little influence (given the values of standardised westedge). The second direction away from $B_{01}$ (second row in above) is roughly equally influenced by X1 and X2 but much less by X3 (which is the N-S direction) and westedge plays a role.

```
cann$Qs[,1]
```

```
        X1         X2         X3
 0.2657933 -0.5034120 -0.8221498
```

Some general scaling occurs with greater influence from X3 (N-S direction), then X2 then X1.

## 7.1 Table of Estimates

```
df <- cbind("diag($B_s$)" = diag(cann$Bs),
      "diag($B_e$)" = diag(cann$Be),
      tQe = t(cann$Qe[-1,-1])) %>%
  as.data.frame()
row.names(df) <- c("$t_2$", "$t_3$")
mykbl <- kbl(df, booktabs = TRUE, position = "!h", escape = FALSE, format = "latex", digits =
  add_header_above(c(" "=3, "$R_e^{\\\\top}$" = 2), escape = FALSE)
mykbl
```

| | | | $R_e^\top$ | |
|---|---|---|---|---|
| | diag($B_s$) | diag($B_e$) | westedge | ones |
| $t_2$ | 0.90 | 0.07 | 0.13 | -0.99 |
| $t_3$ | 0.77 | 0.25 | 0.99 | 0.13 |

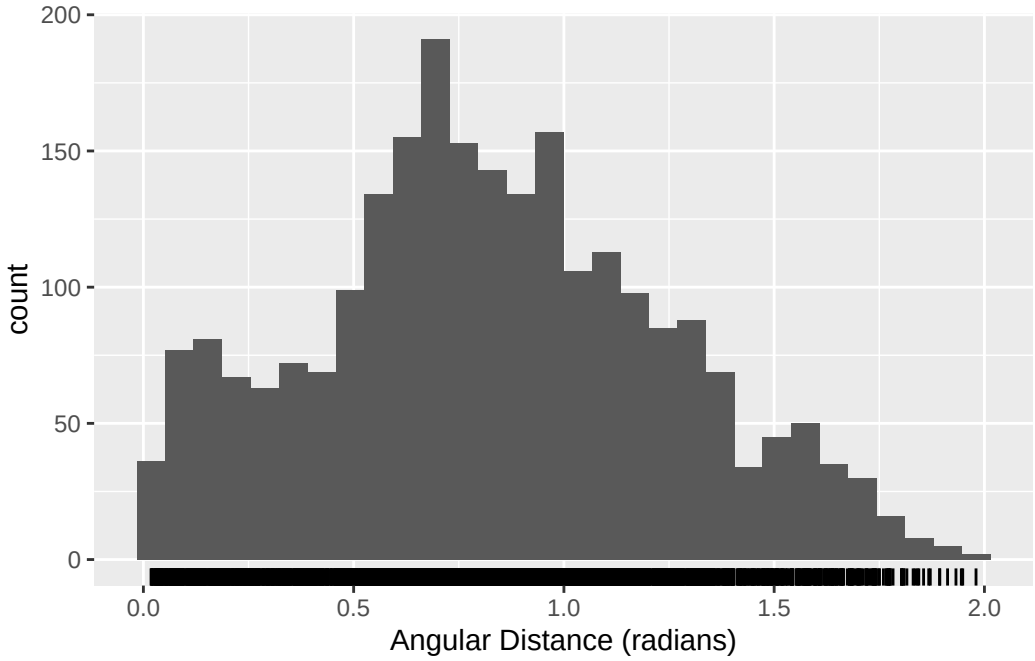
## 7.2 Plot

Note that we have not optimised Q for ESAG2 because optimisation did not converge.


## 8 Angular Distance Between Predictions

```
pred_ang_dist <- acos(pmin(mod$pred %*% t(mod$pred), 1))
tibble::enframe(pred_ang_dist[lower.tri(pred_ang_dist)], value = "ang_dist") %>%
  ggplot() +
  geom_histogram(aes(x = ang_dist), bins = 30) +
```

```
  geom_rug(aes(x = ang_dist)) +
  scale_x_continuous(name = "Angular Distance (radians)")
```




## 9 LOOCV MSE

```
loocvmseSvMF <- function(mod){
  stopifnot(inherits(mod$mean, "mobius_link_Omega"))
  dists <- pbapply::pblapply(1:nrow(mod$y), function(idx){
    newmod <- mobius_SvMF(mod$y[-idx,],
                xs = mod$xs[-idx,],
                xe = mod$xe[-idx,c(-1,-ncol(mod$xe)), drop = FALSE],
                fix_qs1 = FALSE,
                type = "LinEuc",
                G01behaviour = "free",
                mean = mod$mean,
                k = mod$k,
                a = mod$a,
                G0 = mod$G0)
    pred <- mobius_link(xs = mod$xs[idx,, drop = FALSE],
                  xe = mod$xe[idx,, drop = FALSE],
```

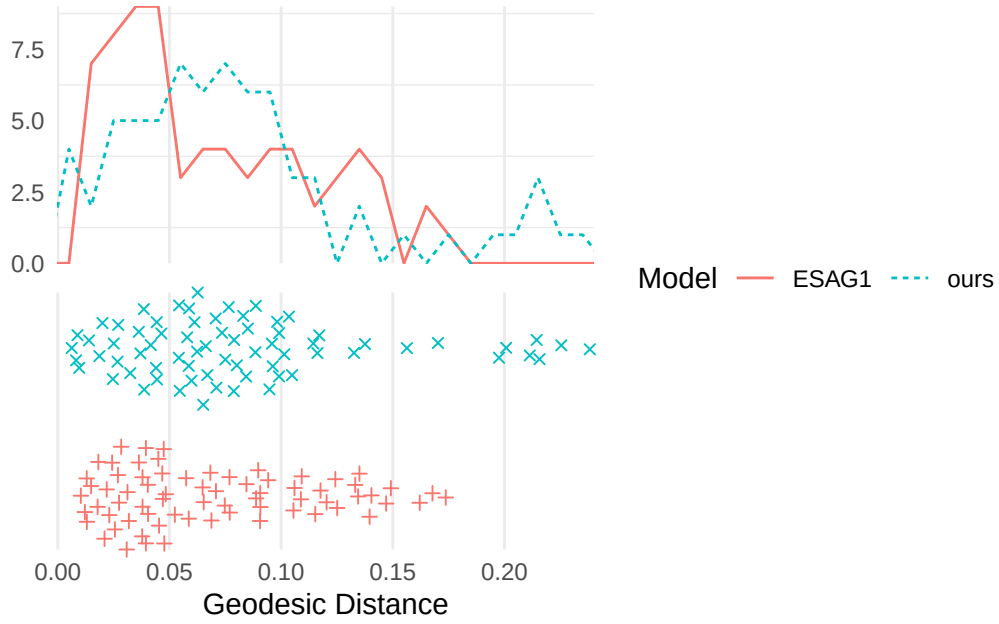
```
                    param = newmod$mean)
    obs <- mod$y[idx,]
    Euc <- vnorm(drop(obs - pred))
    angle <- acos(rowSums(obs * pred))
    return(c(
      Euc = Euc,
      angle = angle
    ))
  })
  dists <- dists %>%
    simplify2array() %>%
    t() %>%
    as_tibble()
  dists %>%
    summarise(across(everything(), ~sum(.x^2)/nrow(mod$y)))
}
loocvmseSvMF(mod)
```

```
# A tibble: 1 x 2
     Euc  angle
   <dbl>  <dbl>
1 0.0107 0.0107
```

Both of these are smaller than the LOOCV MSE that Rosenthal's PLT acheived of 0.074, which corresponds to the Euc metric MSE.

## 10 Residual Size

Geodesic distance between predicted mean and observation ::: {.cell messages='false'} ::: {.cell-




output-display}
::: :::

## 11 DoF

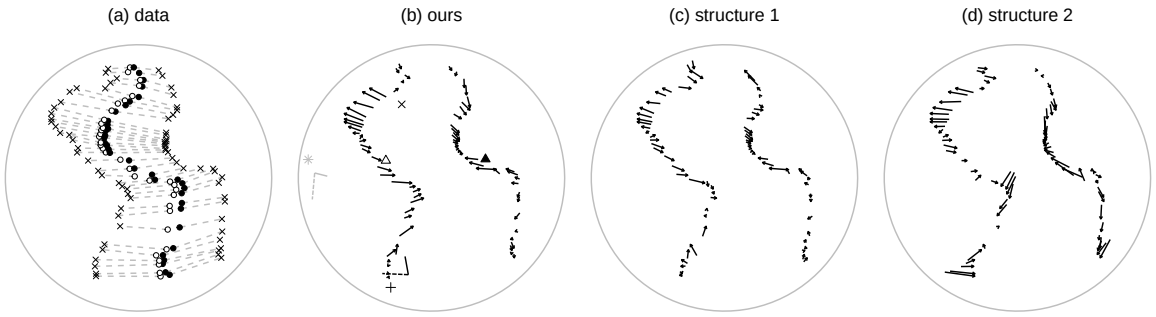
| | Ours |
|---|---|
| | 16 |
| | IAG1 |
| | 17 |
| | ESAG1 |
| | 22 |
| | ESAG2 |
| | 25 |
| Proposition 4(i): spherical only | |
| | 11 |
| Proposition 4(i): spherical + all Euclidean | |
| | 15 |
| Proposition 4(ii): spherical only | |
| | 8 |
| Proposition 4(ii): spherical + all Euclidean | |

## 12 AIC

```
                                     Ours
                                -463.2407
                                     IAG1
                                -370.7254
                                    ESAG1
                                -459.5946
                                    ESAG2
                                -436.2852
            Proposition 4(i): spherical only
                                -184.0309
 Proposition 4(i): spherical + all Euclidean
                                -439.9135
           Proposition 4(ii): spherical only
                                -181.5131
Proposition 4(ii): spherical + all Euclidean
                                -438.5591
```

## 13 Main Figure

```
(plotdata + ggtitle("(a) data") + theme(plot.title = element_text(hjust = 0.5))) +
(plotours + theme(legend.position = "none", plot.title = element_text(hjust = 0.5)) + ggtitle
  (plotESAG1 + ggtitle("(c) structure 1") + theme(plot.title = element_text(hjust = 0.5))) +
  (plotESAG2 + ggtitle("(d) structure 2") + theme(plot.title = element_text(hjust = 0.5))) +
  plot_layout(ncol = 4, widths = c(3,3,3,3))
```



```
ggsave("midatlantic_fig.pdf", width = 12, height = 3.5)
```

Caption: Regression for the midatlantic ridge data. From left: midatlanic ridge (circles) and corresponding locations on the continent (crosses) from Rosenthal el at (2014); our regression; Paine et al structure 1 regression; Paine et al structure 2 regression. The sphere is shown orthogonally projected with north pointing up the page. Arrows: start at the predicted mean and end at the observed continental location, and thus represent residuals. Filled symbols: eastern side. Unfilled symbols: western side. Triangles: Mean of ridge locations and corresponding predicted mean for western or eastern side. Plus symbol: estimated value of $r_{s1}$. Pair of black lines: direction of the estimated second (solid) and third (dashed) columns of $B_0$ located at the estimated first column $b_{01}$ of $B_0$. Pair of grey lines: estimated directions of $\gamma_{02}$ (solid) and $\gamma_{03}$ (dashed) located at the estimated $\gamma_{01}$.

# 14 Appendix

## 14.1 Hessian of Likelihood at Optimum

The parameter vector is longer than the DoF because of the constraints in the optimisation. There should be DoF + `(3-1) * (3 - 2) / 2` positive eigenvalues. The term `(3-1) * (3 - 2) / 2` if for the commutativity constraint on Omega, which the likelihood computation does not account for, so appears as extra degrees of freedom in the Hessian of the likelihood.

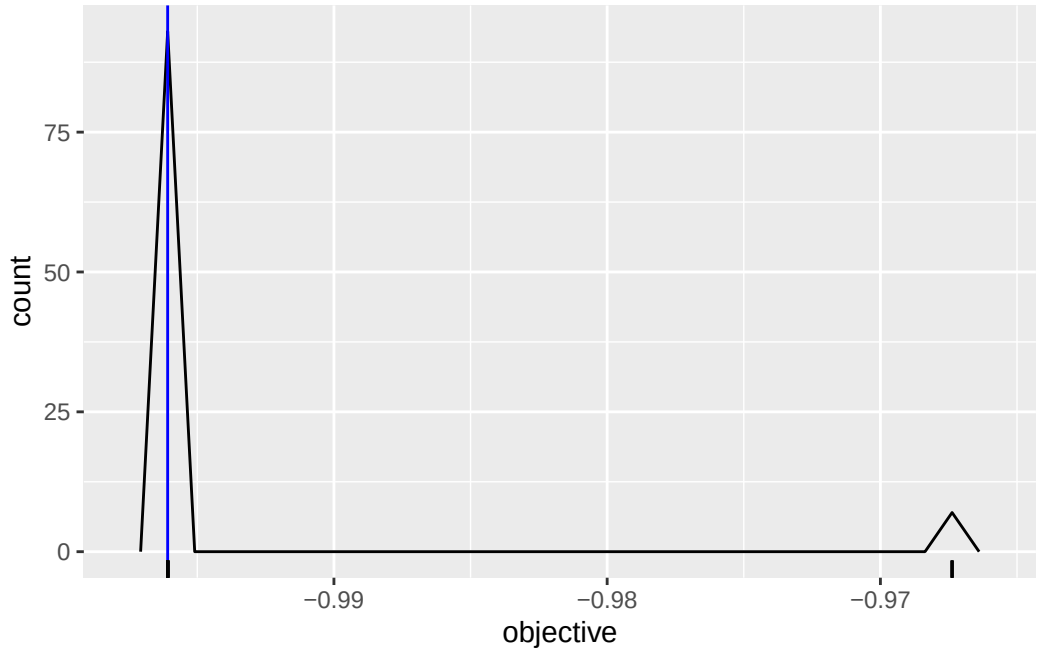
```
 [1] 8.254073e+03 6.672951e+03 2.990692e+03 2.470448e+03 1.236519e+03
 [6] 3.582952e+02 9.508066e+01 2.869560e+01 2.125170e+01 9.630614e+00
[11] 5.113127e+00 4.746740e+00 2.960021e+00 1.951306e+00 1.054189e+00
[16] 4.074127e-01 2.936479e-06
```

These are all positive and non-zero, which confirms that the optimisation routine has found a local maximum of the likelihood.

## 14.2 Check Other Starts for vMF

```
restarts <- pbapply::pblapply(1:100, function(seed){
  start <- rand_mobius_link_cann(p = 3, qs = 3, qe = ncol(xe) + 2, preseed = seed)
  # convert to LinEuc form:
  set.seed(seed+1)
  Qe <- mclust::randomOrthogonalMatrix(ncol(xe)+1, 3-1)
  bigQe <- cbind(0, rbind(0, Qe))
```

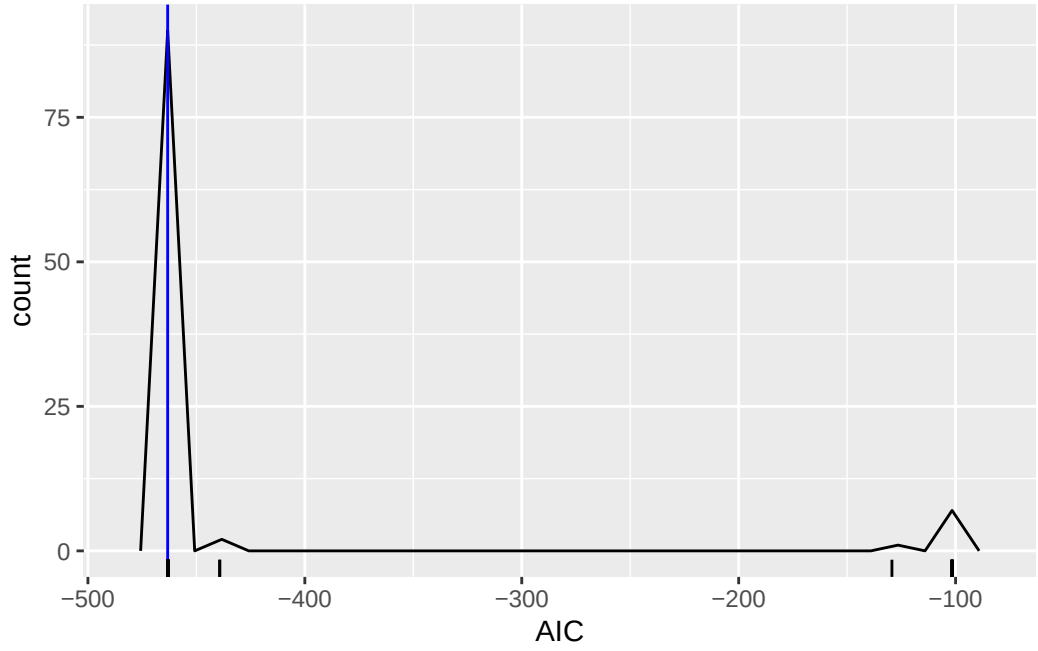
```
  bigQe[, 1] <- 0
  bigQe[1,1] <- 1
  start$Qe <- bigQe
  start$ce <- 1
  modvMF <- mobius_vMF(y = fulldf %>% select(starts_with("Y")) %>% as.matrix(),
      xs = fulldf %>% select(starts_with("X")) %>% as.matrix(),
      xe = xe %>% as.matrix(),
      fix_qs1 = FALSE, type = "LinEuc",
      start = start)
}, cl = 2)
lapply(restarts, "[[", "obj") %>%
  unlist() %>%
  enframe("seed", "objective") %>%
  ggplot()+
  geom_freqpoly(aes(x = objective), bins = 30) +
  geom_vline(xintercept = mod$preest$nlopt$objective, col = "blue") +
  geom_rug(aes(x = objective))
```




Other initial parameters have not improved on the default initial parameters.

### 14.3 Check Other Starts for SvMF

```
restarts <- pbapply::pblapply(1:100, function(seed){
  # randomly generates a SpEuc-form link
  start <- rand_mobius_link_cann(p = 3, qs = 3, qe = ncol(xestd) + 2, preseed = seed)
  # convert to LinEuc form:
  set.seed(seed+1)
  Qe <- mclust::randomOrthogonalMatrix(ncol(xestd)+1, 5-1)
  bigQe <- cbind(0, rbind(0, Qe))
  bigQe[, 1] <- 0
  bigQe[1,1] <- 1
  start$Qe <- bigQe
  start$ce <- 1
  mobius_SvMF(y = fulldf %>% select(starts_with("Y")) %>% as.matrix(),
              xs = fulldf %>% select(starts_with("X")) %>% as.matrix(),
              xe = xestd %>% as.matrix(),
              type = "LinEuc",
              G01behaviour = "free",
              mean = start)
}, cl = 2)
badrestarts <- unlist(lapply(restarts, inherits, "try-error"))
restarts <- restarts[!badrestarts]
lapply(restarts, "[[", "AIC") %>%
  unlist() %>%
  tibble::enframe("seed", "AIC") %>%
  ggplot()+
  geom_freqpoly(aes(x = AIC), bins = 30) +
  geom_vline(xintercept = mod$AIC, col = "blue") +
  geom_rug(aes(x = AIC))
```




Other initial parameters have not improved on the default initial parameters.

### 14.4 Likelihood Ratio Test of vMF vs SvMF

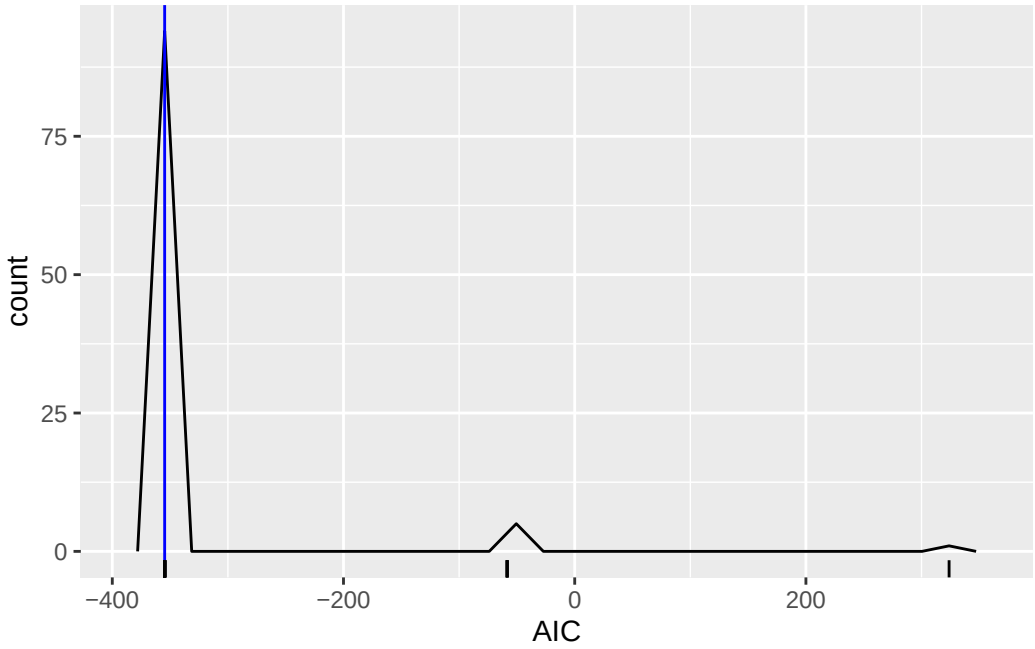
First get best vMF model via a searching with 100 restarts.

```
mod_vMF <- mobius_vMF(y = mod$y,
                      xs = mod$xs,
                      xe = mod$xe,
                      fix_qs1 = FALSE,
                      type = "LinEuc")
mod_vMF_restarts <- pbapply::pblapply(1:100, function(seed){mobius_vMF_refit(mod_vMF, seed)}
```

```
Warning in mobius_vMF_general(y = y, xs = xs, xe = xe, start = start, type =
type, : NLOPT_MAXEVAL_REACHED: Optimization stopped because maxeval (above) was
reached.
```

```
Error in mobius_link_Omega_check(projectedom) :
  The following checks failed. Omega_comm: 0.0043
```

```
lapply(mod_vMF_restarts, "[[", "AIC") %>%
  unlist() %>%
  tibble::enframe("seed", "AIC") %>%
  ggplot()+
  geom_freqpoly(aes(x = AIC), bins = 30) +
  geom_vline(xintercept = mod_vMF$AIC, col = "blue") +
  geom_rug(aes(x = AIC))
```




```
idx <- which.min(lapply(mod_vMF_restarts, "[[", "AIC") %>% unlist())
mod_vMF <- mod_vMF_restarts[[idx]]
mod_vMF$k
```

```
[1] 255.3967
```

```
mod_vMF$AIC
```

```
[1] -354.6918
```

Below is function for comparing likelihoods for responses `y`.

```r
getlikR <- function(y){
  mod1 <- mobius_vMF(y,
                     xs = mod$xs,
                     xe = mod_vMF$xe,
                     type = "LinEuc",
                     fix_qs1 = FALSE,
                     start = mod_vMF$est)
  mod2 <- mobius_SvMF(y,
                      xs = mod$xs,
                      xe = mod_vMF$xe,
                      type = "LinEuc",
                      fix_qs1 = FALSE,
                      G01behaviour = "free",
                      mean = mod$mean,
                      k = mod$k,
                      a = mod$a,
                      G0 = mod$G0)
  if ((mod1$nlopt$status != 4) || (mod2$nlopt$status != 4)){
    return(list(likR = NA_real_,
         vMF = mod1,
         SvMF = mod2))
  }
  lLik1 <- mobius_SvMF_log_lik(mod1$y, xs = mod1$xs, xe = mod1$xe,
              mean = mod1$est,
              k = mod1$k,
              a = rep(1, 3),
              G0 = mod2$G0) %>%
    colSums()
  lLik2 <- mobius_SvMF_log_lik(mod2$y, xs = mod2$xs, xe = mod2$xe,
              mean = mod2$mean,
              k = mod2$k,
              a = mod2$a,
              G0 = mod2$G0) %>%
    colSums()

  list(likR = -2* (lLik1[["R"]] - lLik2[["R"]]),
       vMF = mod1,
       SvMF = mod2)
}
```

Now we simulate the response 1000 times from the best vMF model and compare the likelihood of the vMF model to the likelihood of the SvMF model on each simulated data.

```
null_likRs <- pbapply::pblapply(1:1000, function(seed){
  set.seed(seed)
  y_ld <- rmobius_SvMF(xs = mod$xs,
                       xe = mod_vMF$xe,
                       mean = mod_vMF$mean,
                       k = mod_vMF$k,
                       a = rep(1, 3),
                       G0 = diag(1, 3))
  y <- y_ld[,-4]
  getlikR(y)
}, cl = 2)
obs <- getlikR(mod$y)
```

```
null_likRs_vec <- lapply(null_likRs, "[[", "likR") %>% unlist()
sum(is.na(null_likRs_vec))
```

```
[1] 440
```

```
sum(!is.na(null_likRs_vec))
```

```
[1] 560
```

A substantial fraction of the simulated fits stop at the evaluation limit rather than the convergence tolerance, and these have been discarded.

```
obs$likR
```

```
[1] 116.5489
```

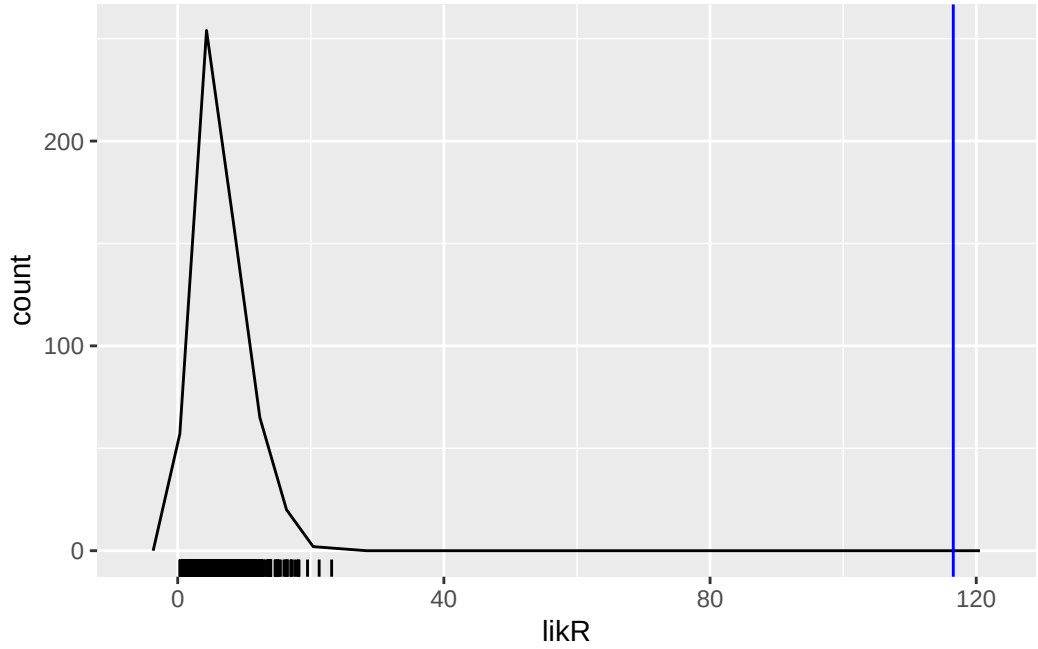
```
range(null_likRs_vec, na.rm = TRUE)
```

```
[1]  0.3178334 23.1321966
```

```
tibble::enframe(null_likRs_vec, "seed", "likR") %>%
  ggplot() +
  geom_freqpoly(aes(x = likR), bins = 30) +
  geom_vline(xintercept = obs$likR, col = "blue") +
  geom_rug(aes(x = likR))
```

```
Warning: Removed 440 rows containing non-finite outside the scale range
(`stat_bin()`).
```

```
Warning: Removed 440 rows containing missing values or values outside the scale range
(`geom_rug()`).
```




p-value is the probability, under the null, of getting a likelihood-ratio that is at least as large as the observed ratio:

```
mean(null_likRs_vec > obs$likR, na.rm = TRUE)
```

```
[1] 0
```

The observed likelihood ratio is far larger than any value obtained under the null, so this small $p$-value suggests that the data was not drawn from a vMF regression.

### 14.4.1 Including the non-converged replicates

Discarding replicates is only safe if the discarded ones could not have changed the conclusion. We can check this directly rather than assume it, because `getlikR()` returns the fitted `vMF` and `SvMF` objects even when it reports `likR = NA`. Those objects hold the parameters at whichever

point the optimiser stopped, so the likelihood ratio can be evaluated for every replicate without refitting anything.

```
likR_from_fits <- function(entry){
  mod1 <- entry$vMF
  mod2 <- entry$SvMF
  lLik1 <- mobius_SvMF_log_lik(mod1$y, xs = mod1$xs, xe = mod1$xe,
              mean = mod1$est, k = mod1$k, a = rep(1, 3), G0 = mod2$G0) %>%
    colSums()
  lLik2 <- mobius_SvMF_log_lik(mod2$y, xs = mod2$xs, xe = mod2$xe,
              mean = mod2$mean, k = mod2$k, a = mod2$a, G0 = mod2$G0) %>%
    colSums()
  -2 * (lLik1[["R"]] - lLik2[["R"]])
}
likR_all <- vapply(null_likRs, likR_from_fits, numeric(1))
```

The vMF fit is the null model and converges for every replicate; only the SvMF fit ever stops early, and always at the evaluation limit:

```
status_vMF <- vapply(null_likRs, function(e) e$vMF$nlopt$status, numeric(1))
status_SvMF <- vapply(null_likRs, function(e) e$SvMF$nlopt$status, numeric(1))
table(vMF = status_vMF, SvMF = status_SvMF)
```

```
   SvMF
vMF   4   5
  4 552 448
```

The two groups have almost the same spread:

```
tapply(likR_all, ifelse(status_SvMF == 4, "converged", "stopped early"), range)
```

```
$converged
[1]  0.2812075 23.1321965

$`stopped early`
[1]  0.4407624 25.8468222
```

Taking every replicate, converged or not, the largest likelihood ratio obtained under the null is still far below the observed one, so the $p$-value is unchanged:

```
max(likR_all)
```

```
[1] 25.84682
```

```
obs$likR
```

```
[1] 116.5489
```

```
mean(likR_all > obs$likR)
```

```
[1] 0
```

These are the ratios at whichever point the optimiser stopped, so they are only informative about the fully optimised ratios if the latter are close. Rather than assume that, continue every stopped-early fit from where it halted, allowing a ten-fold larger `maxeval`:

```
stopped <- which(status_SvMF == 5)
continued <- pbapply::pblapply(stopped, function(i){
  mod1 <- null_likRs[[i]]$vMF
  mod2 <- null_likRs[[i]]$SvMF
  cont <- mobius_SvMF(mod2$y,
                      xs = mod2$xs,
                      xe = mod2$xe,
                      type = "LinEuc",
                      fix_qs1 = FALSE,
                      G01behaviour = "free",
                      mean = mod2$mean,
                      k = mod2$k,
                      a = mod2$a,
                      G0 = mod2$G0,
                      maxeval = 1E5)
  lLik1 <- mobius_SvMF_log_lik(mod1$y, xs = mod1$xs, xe = mod1$xe,
              mean = mod1$est, k = mod1$k, a = rep(1, 3), G0 = cont$G0) %>%
    colSums()
  lLik2 <- mobius_SvMF_log_lik(cont$y, xs = cont$xs, xe = cont$xe,
              mean = cont$mean, k = cont$k, a = cont$a, G0 = cont$G0) %>%
    colSums()
  c(status = cont$nlopt$status,
    extra_iters = cont$nlopt$iterations,
```

```
    likR_after = -2 * (lLik1[["R"]] - lLik2[["R"]]))
}, cl = 2)
continued <- do.call(rbind, continued)
table(status = continued[, "status"])
```

```
status
  4
448
```

```
summary(continued[, "extra_iters"])
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  57.00   75.00   80.00   87.01   84.00 2810.00
```

Every one of them converges, mostly within about eighty further iterations. The likelihood ratio usually barely moves, but not always:

```
summary(continued[, "likR_after"] - likR_all[stopped])
```

```
     Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
-0.941587 -0.000113  0.004862  0.231656  0.033197 16.027048
```

The median change is under 0.01, yet one replicate rises by 16, so a sample of a dozen or so would not have been enough to characterise this. Substituting the continued values gives a null distribution in which every replicate has converged, so nothing has to be discarded at all:

```
likR_final <- likR_all
likR_final[stopped] <- continued[, "likR_after"]
range(likR_final)
```

```
[1]  0.2812075 25.8483420
```

```
obs$likR
```

```
[1] 116.5489
```

```r
mean(likR_final > obs$likR)
```

```
[1] 0
```

The largest likelihood ratio under the null is still far below the observed one, so the $p$-value is zero on the full set of 1000 replicates and the conclusion does not depend on the discarding.